

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	R. Uday kumar Reddy	Reg. No.:	192424190
Section / Batch:	SLOT-C	Date:	22/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Q27. Smart City Waste Collection Routing (Graph Optimization)

A municipality must optimize waste collection routes to minimize fuel consumption and time. Analyze how graph algorithms can be used to determine optimal routes. Identify constraints such as traffic conditions and vehicle capacity. Design a solution using shortest path or heuristic methods. Discuss scalability in large urban environments. Evaluate computational efficiency. Interpret results in terms of cost savings and environmental impact.

Code of Conduct

I R. Uday Kumar Reddy (192424190)____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R. Uday_____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Smart City Waste Collection Routing

1. Problem Overview

Municipal waste collection is a recurring, large-scale logistics operation: a fleet of trucks, each with a limited capacity, must visit a set of collection points (bins/streets) scattered across a city and return to a depot, while minimising total travel time, fuel consumption, and CO2 emissions. This is naturally modelled as a problem on a weighted graph and is closely related to two classical algorithmic problems studied in Design and Analysis of Algorithms: the Shortest Path Problem and the Capacitated Vehicle Routing Problem (CVRP), the latter being a generalisation of the NP-hard Travelling Salesman Problem (TSP).

2. Graph Modelling

The city road network is represented as a weighted graph $G = (V, E)$:

- V (Vertices): depot (single source/sink) and waste collection points (bins, streets, or zones).
- E (Edges): road segments connecting locations, or, in an abstracted model, direct travel links between every pair of points.
- Weight $w(u, v)$: travel cost between u and v — typically distance $\times$ a traffic congestion multiplier, so the weight represents travel time rather than raw distance.

This abstraction lets classical graph algorithms (Dijkstra, Floyd–Warshall, A^*) be reused directly for route computation, while the vehicle-capacity restriction is layered on top as a combinatorial constraint.

3. Constraints

Constraint	Modelling Approach
Traffic conditions	Dynamic/time-dependent edge weights (congestion multiplier applied to base distance); can be refreshed periodically from live traffic feeds.
Vehicle capacity	Each bin has a demand (waste volume); a route's cumulative load must not exceed truck capacity Q — converts the problem into CVRP.
Number of vehicles	Multiple depot-to-depot routes are built; unvisited nodes are assigned to new vehicles once capacity is exhausted.
Time windows (optional)	Bins with fixed collection schedules impose earliest/latest visit times, extending the model to VRPTW.
Road directionality	Directed edges for one-way streets; asymmetric distance matrix.

4. Algorithm Design

4.1 Shortest Path Layer

Dijkstra's algorithm computes the shortest (least-time) path between the depot and every bin, and between every pair of bins, producing an all-pairs travel-time matrix. For non-negative, traffic-adjusted weights this runs in $O((V + E) \log V)$ per source using a min-heap, or $O(V^2)$ for dense graphs using

Floyd–Warshall for all pairs simultaneously in $O(V^3)$. This matrix becomes the input distance metric for the routing layer.

4.2 Routing Layer (Heuristic — CVRP)

Because CVRP is NP-hard, exact solutions (branch-and-bound, dynamic programming over subsets) are only tractable for small instances (roughly $V \leq 15\text{--}20$). For realistic city sizes a constructive heuristic followed by local-search improvement is used:

- Construction: Nearest-Neighbour heuristic — repeatedly extend a route to the closest unvisited bin whose demand still fits the remaining truck capacity; start a new route/vehicle when no bin fits.
- Improvement: 2-opt local search — repeatedly reverse a segment of a route if doing so shortens total travel time, removing path crossings.
- Alternative heuristics: Clarke–Wright Savings algorithm, Genetic Algorithms, Simulated Annealing, or Ant Colony Optimization for larger/harder instances.

This two-layer design (exact shortest-path + heuristic vehicle routing) is the standard pattern used in real-world fleet routing systems (e.g., Google OR-Tools, city waste-management platforms).

5. Implementation

The following Python program builds a synthetic city graph (1 depot + 12 bins), computes all-pairs shortest travel times with Dijkstra, constructs capacitated routes with the nearest-neighbour heuristic, improves them with 2-opt, and estimates fuel cost and CO2 savings.

```
import heapq, random, math

random.seed(42)

# -----
# 1. GRAPH MODEL
# Nodes = depot (0) + waste collection bins (1..N)
# Edge weight = travel time (minutes), influenced by distance + traffic factor
# -----

def build_city_graph(num_bins=12, area=50):
    nodes = {0: (25, 25)} # depot at center
    for i in range(1, num_bins + 1):
        nodes[i] = (random.uniform(0, area), random.uniform(0, area))

    edges = {i: {} for i in nodes}
    for i in nodes:
        for j in nodes:
            if i != j:
                dx = nodes[i][0] - nodes[j][0]
                dy = nodes[i][1] - nodes[j][1]
                dist = math.hypot(dx, dy)
```

```

        traffic_factor = random.uniform(1.0, 1.6) # congestion multiplier
        edges[i][j] = round(dist * traffic_factor, 2)
    return nodes, edges

# -----
# 2. DIJKSTRA - SHORTEST PATH (single source)
# -----
def dijkstra(edges, src, n):
    dist = {node: math.inf for node in range(n)}
    dist[src] = 0
    pq = [(0, src)]
    visited = set()
    while pq:
        d, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        for v, w in edges[u].items():
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    return dist

def all_pairs_shortest_paths(edges, n):
    return {s: dijkstra(edges, s, n) for s in range(n)}

# -----
# 3. CAPACITATED VEHICLE ROUTING (heuristic)
# Nearest-Neighbor construction + capacity constraint,
# then 2-opt local improvement per route
# -----
def nearest_neighbor_cvrp(dist_matrix, demands, depot, capacity, num_bins):
    unvisited = set(range(1, num_bins + 1))
    routes = []
    while unvisited:
        route = [depot]
        load = 0
        current = depot
        while True:
            candidates = [b for b in unvisited if load + demands[b] <= capacity]
            if not candidates:
                break
            nxt = min(candidates, key=lambda b: dist_matrix[current][b])
            route.append(nxt)
            load += demands[nxt]
            unvisited.remove(nxt)

```

```

        current = nxt
        route.append(depot)
        routes.append((route, load))
    return routes

def route_length(route, dist_matrix):
    return sum(dist_matrix[route[i]][route[i+1]] for i in range(len(route)-1))

def two_opt(route, dist_matrix):
    best = route[:]
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best) - 2):
            for j in range(i + 1, len(best) - 1):
                new_route = best[:i] + best[i:j+1][::-1] + best[j+1:]
                if route_length(new_route, dist_matrix) < route_length(best, dist_matrix):
                    best = new_route
                    improved = True
    return best

# -----
# 4. RUN SIMULATION
# -----

NUM_BINS = 12
CAPACITY = 100
nodes, edges = build_city_graph(NUM_BINS)
n = NUM_BINS + 1
apsp = all_pairs_shortest_paths(edges, n)

demands = {i: random.randint(10, 30) for i in range(1, NUM_BINS + 1)}
demands[0] = 0

routes = nearest_neighbor_cvrp(apsp, demands, 0, CAPACITY, NUM_BINS)

print("=== Shortest Travel-Time Matrix (Dijkstra, sample depot row) ===")
print({k: v for k, v in list(apsp[0].items())[6]}, "...")

print("\n=== Initial CVRP Routes (Nearest Neighbor, capacity =", CAPACITY, ") ===")
total_before = 0
for idx, (route, load) in enumerate(routes, 1):
    length = route_length(route, aosp)
    total_before += length
    print(f"Truck {idx}: Route {route} Load={load} Time={length:.2f} min")

print(f"\nTotal travel time BEFORE optimization: {total_before:.2f} min")

```

```

print("\n=== After 2-opt Local Optimization ===")
total_after = 0
optimized_routes = []
for route, load in routes:
    opt_route = two_opt(route, apsp)
    length = route_length(opt_route, apsp)
    total_after += length
    optimized_routes.append((opt_route, load, length))
    print(f'Route {opt_route} Load={load} Time={length:.2f} min")

print(f'\nTotal travel time AFTER optimization: {total_after:.2f} min")
saving_pct = (total_before - total_after) / total_before * 100
print(f'Improvement: {saving_pct:.2f}%")

# -----
# 5. Environmental / cost impact estimation
# -----
FUEL_RATE = 0.08 # litres per minute of travel (assumed avg for waste truck)
FUEL_COST = 100 # currency units per litre
CO2_PER_LITRE = 2.68 # kg CO2 per litre diesel

fuel_before = total_before * FUEL_RATE
fuel_after = total_after * FUEL_RATE
cost_before = fuel_before * FUEL_COST
cost_after = fuel_after * FUEL_COST
co2_before = fuel_before * CO2_PER_LITRE
co2_after = fuel_after * CO2_PER_LITRE

print("\n=== Cost & Environmental Impact ===")
print(f'Fuel used before: {fuel_before:.2f} L | after: {fuel_after:.2f} L")
print(f'Fuel cost before: {cost_before:.2f} | after: {cost_after:.2f}")
print(f'CO2 emitted before: {co2_before:.2f} kg | after: {co2_after:.2f} kg")
print(f'Fuel/Cost/CO2 reduction: {saving_pct:.2f}%")

```

6. Output

```

=== Shortest Travel-Time Matrix (Dijkstra, sample depot row) ===
{0: 0, 1: 38.97, 2: 21.44, 3: 15.58, 4: 30.13, 5: 35.53} ...

=== Initial CVRP Routes (Nearest Neighbor, capacity = 100 ) ===
Truck 1: Route [0, 8, 3, 11, 9, 6, 0] Load=98 Time=85.37 min
Truck 2: Route [0, 2, 12, 5, 1, 10, 0] Load=95 Time=96.80 min
Truck 3: Route [0, 4, 7, 0] Load=33 Time=116.40 min

```

Total travel time BEFORE optimization: 298.57 min

=== After 2-opt Local Optimization ===

Route [0, 9, 6, 11, 3, 8, 0] Load=98 Time=82.02 min

Route [0, 2, 12, 5, 1, 10, 0] Load=95 Time=96.80 min

Route [0, 4, 7, 0] Load=33 Time=116.40 min

Total travel time AFTER optimization: 295.22 min

Improvement: 1.12%

=== Cost & Environmental Impact ===

Fuel used before: 23.89 L | after: 23.62 L

Fuel cost before: 2388.56 | after: 2361.76

CO2 emitted before: 64.01 kg | after: 63.30 kg

Fuel/Cost/CO2 reduction: 1.12%

7. Result Interpretation

Metric	Before Optimization	After 2-opt	Improvement
Total travel time	298.57 min	295.22 min	1.12%
Fuel consumed	23.89 L	23.62 L	1.12%
Fuel cost	2388.56 units	2361.76 units	1.12%
CO2 emitted	64.01 kg	63.30 kg	1.12%

On this small 12-bin instance the 2-opt pass yields a modest ~1% improvement because the nearest-neighbour construction was already fairly good for a small node set; on real city-scale instances (hundreds of bins), 2-opt / Clarke–Wright / metaheuristics typically cut route length by 10–25% over a naive nearest-neighbour or manually-drawn route, since crossing paths and inefficient detours compound with scale. Even a 10% reduction in travel time translates directly into proportional fuel savings, lower operating cost, and reduced CO2 output, plus secondary benefits: less road congestion, less noise pollution, and lower truck wear-and-tear.

8. Scalability in Large Urban Environments

- **Precomputation:** All-pairs shortest paths for the whole city are expensive ($O(V^3)$ via Floyd–Warshall); in practice, only the depot-to-bin and bin-to-bin distances relevant to a day's collection zone are computed, or a road-network shortest-path index (e.g., Contraction Hierarchies, A* with landmarks) is precomputed once and queried in near-constant time.
- **Zoning / clustering:** The city is partitioned into geographic zones (e.g., via k-means or Voronoi partitioning around depots) so that CVRP is solved independently per zone — this bounds problem size per sub-instance and enables parallel computation.
- **Dynamic traffic:** Edge weights are refreshed from live traffic APIs at intervals; re-routing uses incremental updates rather than recomputing the full graph.
- **Heuristic choice at scale:** Nearest-neighbour + 2-opt scales to hundreds of nodes; for thousands, metaheuristics (Genetic Algorithms, Tabu Search, Ant Colony Optimization) or commercial solvers (Google OR-Tools, which uses guided local search) are preferred.

- Parallelization: Route construction and 2-opt improvement per truck are independent and can be distributed across cores/machines.

9. Computational Efficiency

Component	Algorithm	Time Complexity
Single-source shortest path	Dijkstra (min-heap)	$O((V + E) \log V)$
All-pairs shortest path	Floyd–Warshall	$O(V^3)$
Route construction	Nearest Neighbour	$O(V^2)$ per vehicle
Route improvement	2-opt	$O(V^2)$ per pass, repeated until no improvement
Exact CVRP (small V)	DP over subsets / Branch-and-Bound	$O(2^V \cdot V^2)$ — intractable beyond ~ 20 nodes

The overall pipeline (Dijkstra + Nearest-Neighbour + 2-opt) is polynomial and practical for real deployments, trading a small amount of optimality (heuristics do not guarantee the global optimum, unlike exact CVRP solvers) for tractable running time at city scale — a necessary trade-off since exact CVRP is NP-hard.

10. Conclusion

Modelling waste collection as a graph problem allows classical, well-understood algorithms to be combined into a practical system: Dijkstra/Floyd–Warshall for accurate travel-time estimation under traffic, and capacity-aware heuristics (Nearest-Neighbour + 2-opt, or more advanced metaheuristics at scale) for vehicle routing. The approach is polynomial-time, scales to large cities through zoning and precomputed road-network indices, and directly reduces fuel cost and carbon emissions — even single-digit percentage gains in route efficiency compound into significant municipal savings and environmental benefit when applied across an entire city fleet, every day, over a year.